

Webpack 篇

1、说说你对 webpack 的理解？解决了什么问题？

把我们写的代码转为浏览器可以执行的代码：

Webpack 是一个现代 JavaScript 应用程序的模块打包工具。它的核心功能是将项目中的各种资源（JavaScript、CSS、图片等）视为模块，然后通过一系列加载器和插件，将这些模块打包为可以在浏览器中运行的文件（通常是一个或多个 JavaScript 文件）。

模块化开发：Webpack 支持各种模块化标准（如 ES Modules、CommonJS、AMD），使得开发者可以更好地组织和管理代码。模块化的代码更易于维护、调试和重用。

依赖关系管理：Webpack 通过分析项目中的依赖关系，自动处理模块之间的依赖，避免了手动管理依赖的麻烦，同时确保了模块加载的正确顺序。

文件打包和优化：Webpack 能够将项目中的多个文件（包括 JS、CSS、图片等）打包成一个或多个文件，从而减少了 HTTP 请求数量，提升了页面加载性能。此外，它还支持代码压缩、图片压缩、CSS 提取等优化操作。

代码分割：Webpack 支持代码分割功能，可以将应用程序拆分成多个块（chunks），并按需加载。这种方式可以减少初始加载时间，并提升应用程序的响应速度。

热模块替换：Webpack 提供了热模块替换功能，使得在开发过程中可以实时更新已修改的模块，而不需要刷新整个页面，从而提升了开发效率。

处理非 JavaScript 资源：Webpack 不仅可以处理 JavaScript，还可以通过使用 loader 处理其他类型的文件，例如将 SCSS 或 LESS 编译为 CSS，将 TypeScript 转换为 JavaScript，将图片文件转换为 base64 等。

多环境支持：Webpack 可以根据配置的不同，生成适合开发、生产等不同环境的代码。例如，开发环境下可以保留调试信息，而生产环境下可以进行代码压缩和混淆。

webpack 的核心能力：

1、打包压缩：

2、开发服务器：(npm run build) 最开始需要打包之后在看效果，npm run serve：启动开发服务器（本地）

1、浏览器请求开发服务器

2、开发服务器从内存中拿到打包结果交给浏览器

3、当源码发生变化时，webpack 监听源码目录（重新打包），再去刷新浏览器页面

4、浏览器页面刷新继续请求开发服务器，开发服务器继续从内存中拿到打包结果。

3、文件指纹：涉及浏览器和服务器的关系，浏览器缓存服务器发过来的资源文件，但当（服务器更新文件）文件发生变化时，浏览器不能监听到，因此产生文件指纹功能，只要文件内容发生变化，文件的命名就发生变化。

减少请求和使用最新的代码文件

在 package.json 文件中存在 analyse 指令，运行 npm analyse 就可以在 build 文件夹中看到打包之后的结果分析`

2、说说 webpack 的构建流程？

1. 初始化阶段：Webpack 从配置文件（如 webpack.config.js）读取并合并用户的配置，然后创建一个 Compiler 实例。这个实例中包含了所有的配置信息和钩子（生命周期函数），接下来它会开始整个构建过程。

2. 编译阶段：Webpack 会从入口点开始递归地解析每个模块的依赖关系。每找到一个新的模块，就会根据模块类型和 loader 规则进行处理。处理后的模块会被加入到依赖图中。

解析入口点：Webpack 根据配置找到入口文件。entry：入口

模块解析与加载：Webpack 使用配置的 loader 处理模块，支持各种资源类型（如 JS、CSS、图片等）。

依赖关系分析：Webpack 通过静态分析，识别模块中引入的依赖，并继续递归地解析这些依赖。

3. 构建阶段：在解析完所有模块的依赖后，Webpack 会开始根据依赖图将模块组合起来。这个过程包括将各个模块转换为浏览器可执行的代码(js 代码)，并根据需要进行优化。

模块合并：Webpack 将模块按照依赖关系打包到一个或多个文件中。

代码分割：根据配置，将代码拆分为多个块，实现按需加载。

4. 优化阶段：在构建完成后，Webpack 会对输出的文件进行一系列优化操作。常见的优化包括：

Tree Shaking：移除未使用的代码。

代码压缩：压缩 JS 和 CSS 文件，减少文件体积。

文件指纹 (Hashing)：为文件生成唯一的哈希值，便于浏览器缓存管理。

代码拆分 (Splitting)：将代码拆分成更小的块，按需加载。

5. 输出阶段：根据入口文件及其依赖关系将其打包成一个或多个 chunk，再把每个 Chunk 转换成一个单独的文件加入到指定的输出列表 (seal 方法生成 chunk)，这步是可以修改输出内容的最后机会。

6. 完成阶段：构建流程完成，Webpack 触发 done 钩子，根据配置确定输出的路径和文件名进行打包。

可以在此阶段进行一些后续处理，如通知开发者构建完成、自动刷新浏览器等。

3、loader 的作用

默认情况下,在遇到 import 或者 load 加载模块的时候,webpack 只支持对 js 文件打包,像 css、sass、png 等这些类型的文件的时候,webpack 则无能为力,这时候就需要配置对应的 loader 进行文件内容的解析。

关于配置 loader 的方式有三种：

配置方式 (推荐): 在 webpack.config.js 文件中指定 loader: 一般配置 test 属性表示匹配的规则和 use 属性表示使用的 loader

内联方式: 在每个 import 语句中显式指定 loader

CLI 方式: 在 shell 命令中指定它们

注意: 因为 loader 支持链式调用, 链中的每个 loader 会处理之前已处理过的资源, 最终变为 js 代码。顺序为相反的顺序执行`

babel-loader: 使用 Babel 将 ES6+ 语法转换为 ES5, 确保代码兼容老旧浏览器。

css-loader 和 style-loader:

作用: css-loader 使你能够使用 @import 和 url() 这样的方式来处理 CSS 文件的 import 语句。

style-loader 则将 CSS 插入到页面的 <style> 标签中, 通过 DOM 操作去加载 CSS。。

使用场景: 当你想要在 JavaScript 中引入并应用 CSS 样式时。一般需要配合使用

file-loader 和 url-loader:

作用: file-loader 用于处理文件导入 (如图片、字体文件等), 将文件输出到构建目录, 并返回文件的 URL。

url-loader 类似, 但在文件大小低于设定值时, 会将文件转换为 Base64 字符串, 嵌入到代码中。

使用场景: 需要引入并处理图片、字体等静态资源时。

sass-loader:less-loader:postcss-loader:

作用: 将 SCSS/SASS 文件转换为 CSS, 通常配合 css-loader 和 style-loader 使用。

使用场景: 在项目中使用 SASS/SCSS 预处理器编写样式时。

ts-loader:

作用: 将 TypeScript 文件转换为 JavaScript 文件。

使用场景: 使用 TypeScript 编写代码时。

html-loader:

导入 HTML 文件中的资源：例如，它可以处理 `<img>` 标签中的图片路径，并确保这些资源被正确打包和引用。

压缩 HTML 结构：虽然 `html-loader` 主要是用来解析和导入 HTML，但它可以与其他插件（如 `html-minifier`）结合使用，进行 HTML 压缩和优化。

`vue-loader`:

作用：将 Vue 单文件组件（.vue 文件）转换为 JavaScript 模块。

使用场景：在项目中使用时使用 Vue.js 框架时。

`html-minify-loader`: 压缩 HTML

4、plugins 的作用

1. HtmlWebpackPlugin

功能：自动生成 HTML 文件，并将打包后的资源（如 CSS 和 JS 文件）注入其中。

解决的问题：

自动化 HTML 文件的生成，避免手动维护。

确保生成的 HTML 文件始终引用最新的资源。

2. MiniCssExtractPlugin

功能：将 CSS 提取为独立的 .css 文件，而不是将其内联在 JavaScript 文件中。

解决的问题：

改善了样式的加载性能，允许浏览器并行加载 CSS 文件。

提高了样式文件的可维护性和可缓存性。

3. CleanWebpackPlugin

功能：在每次构建前清理输出目录（如 `dist` 文件夹）。

解决的问题：

防止旧文件与新文件混淆，确保输出目录的干净整洁。

4. TerserWebpackPlugin

功能：压缩和优化 JavaScript 代码。

解决的问题：

减少 JavaScript 文件的体积，提高加载速度。

去除不必要的代码，如注释和未使用的变量。

5. CssMinimizerPlugin

功能：压缩 CSS 文件。

解决的问题：

减少 CSS 文件的体积，提高加载速度。

6. DefinePlugin

功能：创建全局常量，可以在代码中使用。

解决的问题：

可以根据不同的环境（如开发或生产）定义不同的变量，控制代码的行为。

7. CopyWebpackPlugin

功能：复制静态文件到输出目录。

解决的问题：

方便地将静态资源（如图片、字体）包含在构建中，而无需手动管理文件。

8. HotModuleReplacementPlugin

功能：启用热模块替换（HMR），允许在运行时更新模块，而不需完全刷新页面。

解决的问题：

提高开发效率，提供更快反馈循环。

9. ImageMinimizerPlugin

功能：压缩和优化图像文件。

解决的问题：

减少图像文件的体积，提高网页加载速度。

10. BundleAnalyzerPlugin

功能：可视化打包后的文件大小和组成。

解决的问题：

帮助开发者了解哪些文件占用了较多的空间，从而优化打包和依赖。

5、loader 和 plugin 的区别

loader 是文件加载器，能够加载资源文件，并对这些文件进行一些处理，诸如编译、压缩等，最终一起打包到指定的文件中。把 js 和 json 外的其它文件转为 Webpack 可以识别的模块

plugin 赋予了 webpack 各种灵活的功能，例如打包优化、资源管理、环境变量注入等，目的是解决 loader 无法实现的其他事

1、执行时机不同：entry=>loaders=>output 而插件可以在整个编译期间都起作用

Webpack 运行的生命周期中会广播出许多事件，Plugin 可以监听这些事件，在合适的时机通过 Webpack 提供的 API 改变输出结果

对于 loader，实质是一个转换器，将 A 文件进行编译形成 B 文件，操作的是文件，比如将 A.scss 或 A.less 转变为 B.css，单纯的文件转换过程

2、配置方式不同：loader 通过 module.rules 数组配置，plugins 数组中配置。每个插件通常是一个实例化后的对象，并可以通过构造函数传递选项。

6、webpack 的生命周期

compiler.initialize: 初始化。

compiler.run: 开始构建。

compiler.compilation: 创建编译实例。

compiler.make: 开始一次编译（每个文件编译时会触发，包括下述红色部分）。

compilation.buildModule: 构建模块。

compilation.seal: 构建完成。

compilation.optimize: 模块优化。

compiler.afterCompile: 所有文件编译完成。

compiler.emit: 输出资源。

compiler.done: 构建过程完成。

7、webpack 的热更新是如何做到的？原理是什么？

我们在应用运行过程中修改了某个模块，通过自动刷新会导致整个应用的整体刷新，那页面中的状态信息都会丢失

如果使用的是 HMR，就可以实现只将修改的模块实时替换至应用中，不必完全刷新整个应用

如果我们修改并保存 css 文件，确实能够以不刷新的形式更新到页面中，但是，当我们修改并保存 js 文件之后，页面依旧自动刷新了，这里并没有触发热模块

所以，HMR 并不像 Webpack 的其他特性一样可以开箱即用，需要有一些额外的操作。们需要去指定哪些模块发生更新时进行 HRM，如下代码：

```
if(module.hot){  
  
    module.hot.accept('./util.js',()=>=>{
```



```
console.log("util.js 更新了")
```

```
}}
```

```
}
```

启动时建立 WebSocket 连接：在项目启动时，Webpack 会创建与开发服务器的 WebSocket 连接，用于在构建完成后接收更新的模块。

构建编译阶段：Webpack 在编译时会在每个模块中注入 HMR 运行时代码。该代码会监听源代码的变化，并通知 HMR runtime 进行处理。

文件监控：Webpack 会监控所有入口文件及其依赖的文件，一旦检测到文件发生变化，会触发重新编译。

构建完成：当编译完成后，Webpack 会将编译结果发送给开发服务器，并通过 WebSocket 通知客户端有新的资源可用。

模块更新：在客户端收到更新通知后，HMR runtime 会与服务器建立一条新的短连接，并请求更新的模块信息。

应用更新：服务器端收到模块请求后，会以增量更新的方式构建新的模块，并通过短连接发送给客户端。

模块替换：在客户端收到新的模块后，HMR runtime 会根据更新策略决定如何替换旧的模块。一般情况下，它会尝试保留应用的状态，例如保留表单数据、滚动位置等。

8、说说如何借助 webpack 来优化前端性能？

1.JS 代码压缩： TerserPlugin 插件

2.CSS 代码压缩：css-minimizer-webpack-plugin：CSS 压缩通常是去除无用的空格等，

3.Html 文件代码压缩：使用 HtmlWebpackPlugin 插件来生成 HTML 的模板时候，通过配置属性 minify 进行 html 优化

```
module.exports = {
```

```

plugin:[

  new HtmlWebpackPlugin({

    minify:{

      minifyCSS:false, // 是否压缩 css

      collapseWhitespace:false, // 是否折叠空格

      removeComments:true // 是否移除注释

    }

  }]
]

```

} 设置了 minify，实际会使用另一个插件 html-minifier-terser 4. 文件大小压缩 (gzip 压缩)：

`compression-webpack-plugin`，对文件的大小进行压缩，减少 http 传输过程中宽带的损耗 5. 图片压缩：一般在 loader 里面配置：image-webpack-loader

Tree Shaking：

1、usedExports：通过标记某些函数是否被使用，之后通过 Terser 来进行优化的

2、sideEffects 用于告知 webpack compiler 哪些模块时有副作用，配置方法是在 package.json 中设置 sideEffects 属性

css tree shaking：`purgecss-plugin-webpack` 插件 6. 代码分离：将代码分离到不同的 bundle 中，之后我们可以按需加载，或者并行加载这些文件，默认情况下，所有的 JavaScript 代码（业务代码、第三方依赖、暂时没有用到的模块）在首页全部都加载，就会影响首页的加载速度

新的 chunk 是否被共享或者是来自 node_modules 的模块；

新的 chunk 体积在压缩之前是否大于 30kb；

按需加载 chunk 的并发请求数量小于等于 5 个；

页面初始加载时的并发请求数量小于等于 3 个；

代码分离可以分出更小的 bundle，以及控制资源加载优先级，提供代码的加载性能，这里通过 `splitChunksPlugin` 来实现，该插件 webpack 已经默认安装和集成，只需要配置即可。(一般针对异步请求) 7.内联 chunk:可以通过 InlineChunkHtmlPlugin 插件将一些 chunk 的模块内联到 html，如 runtime 的代码（对模块进行解析、加载、模块信息相关的代码），代码量并不大，但是必须加载的

9、如何提高 webpack 的构建速度？

升级最新版本

优化 loader 配置：使用 loader 时，可以通过配置 include、exclude、test 属性来匹配文件，接触 include、exclude 规定哪些匹配应用 loader

合理使用 resolve.extensions：resolve.extensions 是解析到文件时自动添加拓展名，不要随便把所有后缀都写在里面，这会调用多次文件的查找，这样就会减慢打包速度。

优化 resolve.modules：

优化 resolve.alias：alias 给一些常用的路径起一个别名

使用 DLLPlugin 插件：避免重复编译第三方库

使用 cache-loader：在一些性能开销较大的 loader 之前添加 cache-loader，以将结果缓存到磁盘里，显著提升二次构建速度

terser 启动多线程：使用多进程并行运行来提高构建速度

合理使用 sourceMap：

服务器与服务器之间请求数据并不会存在跨域行为，跨域行为是浏览器安全策略限制`

10、生产环境和开发环境

开发环境：开发人员调试开发的一种环境：方便调试，保持高效的开发：开发环境调优的目的是提升

webpack 构建速度。如开启热更新和热模块（HMR）替换、缩小代码编译范围、多线程打包等

生产环境：发布上线的环境：让程序在生产环境中正常有效的运行：利用缓存、减少代码体积、分包等方式

提升页面响应速度。对于 spa 单页的话通常需要通过优化减少首屏渲染时间

开发环境的需求：

模块热更新（本地开启服务，实时更新）

sourceMap (代码映射，方便打包调试)

接口代理（配置 proxyTable 解决开发环境中的跨域问题）

代码规范检查（代码规范检查工具）

生产环境的需求：

提取公共代码

压缩混淆(压缩混淆代码，清除代码空格，注释等信息使其变得难以阅读)

文件压缩/base64 编码(压缩代码，减少线上环境文件包的大小)

去除无用的代码

开发环境和生产环境的共同需求：

入口

代码处理(loader 处理)

解析配置

11、webpack 的核心特性

1、模块打包：将项目中的各种模块（JavaScript、CSS、图像等）打包成一个或多个模块化的文件。它可以处理各种类型的资源（包括非 JavaScript 文件），并通过加载器将它们转化为 Webpack 可以理解的模块。

2、代码分割：将代码按需加载（lazy loading）或动态加载，这样可以优化加载性能，减少首屏加载时间。

Webpack 会自动将代码分割成多个文件（chunks），根据需求进行加载。

3、依赖图：递归地解析应用程序中的所有模块及其依赖关系，构建出一个依赖图。通过这个依赖图，Webpack 确定如何将模块组合到一起，以生成最优化的输出文件

4、加载器：使用加载器来处理非 JavaScript 文件，如 CSS、图片、字体等。加载器是 Webpack 中的一个转换器，可以将各种类型的文件转换为可以被 Webpack 打包的模块。

例如，使用 `css-loader` 可以处理 CSS 文件，使用 `file-loader` 可以处理图片文件等。

5、插件系统：允许自定义构建过程和功能扩展。插件可以用来优化打包、压缩文件、环境变量设置等。

6、Tree Shaking:它可以移除未使用的代码，从而减少打包文件的大小。它主要依赖于 ES6 模块的静态结构来分析哪些代码在最终的打包中是未使用的。

7、热模块替换:允许在开发过程中动态替换、添加或移除模块，而无需完全刷新页面。这提高了开发效率，尤其是前端开发中调试和测试的速度

8、开发服务器:一个开发服务器 `webpack-dev-server`，它可以实时监听代码变动并自动刷新页面或者进行热模块替换，方便前端开发调试

html 篇

1、html 基本结构和元素介绍

`<!DOCTYPE html>`：告诉浏览器（解析器）应该以什么样（html 或 xhtml）的文档类型定义来解析文档，不同的渲染模式会影响浏览器对 CSS 代码甚至 JavaScript 脚本的解析。它必须声明在 HTML 文档的第一行。

`就是让浏览器进入标准模式(严格模式),使用最新的 HTML5 标准来解析渲染页面;如果不写,浏览器就会进入混杂模式,我们需要避免此类情况发生。`

浏览器的渲染模式: document.compatMode

CSS1Compact:标准模式,默认模式,浏览器使用 W3C 的标准解析渲染页面。BackCompat:怪异模式

<html lang="en">: 定义 HTML 文档的根元素, lang="en" 属性指定文档的语言为英语。

<head>:标签用于定义文档的头部,它是所有头部元素的容器。 head 中的元素可以引用脚本、指示浏览器在哪里找到样式表、提供元信息等。包含文档的元数据 (meta-data)、样式表、脚本等。

`元数据在浏览器加载页面之前进行解析,用于设置文档的字符集、描述、关键词、作者、视口设置等,影响页面的展示和行为。`

<meta>:由 name 和 content 属性定义,用来描述网页文档的属性,比如网页的作者,网页描述,关键词等,除了 HTTP 标准固定了一些 name 作为大家使用的共识,开发者还可以自定义 name。

<meta name="author" content="Your Name">

<meta name="keywords" content="关键词" />

<meta name="description" content="页面描述内容" />好的关键字和描述有利于搜索

<meta name="copyright" content="Your Company Name">

<meta name="date" content="2024-07-13" scheme="YYYY-MM-DD">

<meta charset="UTF-8">:指定文档的字符编码为 UTF-8。

原因: UTF-8 是一种通用的字符编码,能够表示几乎所有语言的字符,确保文档内容在不同语言环境下都能正确显示。

<meta name="viewport" content="width=device-width, initial-scale=1.0">:控制视口的尺寸和缩放,width=device-width 将视口的宽度设置为设备的宽度, initial-scale=1.0 设置初始缩放比例为 1。

这些设置确保页面在不同设备上都有良好的显示效果,尤其是在 [移动设备] 上。

<title>Document</title>:设置文档的标题。

<body>: 包含文档的主体内容, 是用户可以看到和交互的部分。

原因: <body> 标签包裹了所有呈现在页面上的内容, 包括文本、图像、表格、链接等。所有实际的页面内容都应该放在这里。

2、WEB 标准以及 W3C 的理解

web 标准简单来说可以分为结构、表现和行为。其中结构主要是有 HTML 标签组成。或许通俗点说, 在页面 body 里面我们写入的标签都是为了页面的结构。表现即指 css 样式表, 通过 css 可以是页面的结构标签更具美感。

行为是指页面和用户具有一定的交互, 同时页面结构或者表现发生变化, 主要是由 js 组成。

web 标准一般是将该三部分独立分开, 使其更具有模块化。但一般产生行为时, 就会有结构或者表现的变化, 也使这三者的界限并不那么清晰。

W3C 对 web 标准提出了规范化的要求, 也就是在实际编程中的一些代码规范: 包含如下几点

对于结构要求: (标签规范可以提高搜索引擎对页面的抓取效率, 对 SEO 很有帮助)

- 1) 标签字母要小写
- 2) 标签要闭合
- 3) 标签不允许随意嵌套

对于 css 和 js 来说

- 1) 尽量使用外链 css 样式表和 js 脚本。是结构、表现和行为分为三块, 符合规范。同时提高页面渲染速度, 提高用户的体验。
- 2) 样式尽量少用行间样式表, 使结构与表现分离, 标签的 id 和 class 等属性命名要做到见文知义, 标签越少, 加载越快, 用户体验提高, 代码维护简单, 便于改版

3) 不需要变动页面内容，便可提供打印版本而不需要复制内容，提高网站易用性。

3、行内元素和块级元素有哪些

行内元素：span,button,input,a,strong,em,label,

‘注意：行内元素不能设置和高度有关的属性’

块级元素：div,img,h1-h6,p,ul,ol,li,table,from,header

4、说一下 web worker

在 HTML 页面中，如果在执行脚本时，页面的状态是不可响应的，直到脚本执行完成后，页面才变成可响应。web worker 是运行在后台的 js，独立于其他脚本，不会影响页面的性能。并且通过 postMessage 将结果回传到主线程。这样在进行复杂操作的时候，就不会阻塞主线程了。

如何创建 web worker：

1、主线程创立

const worker = new Worker('worker.js'); 里面放入的是计算脚本的链接

2、worker.postMessage({ key: 'value' });向分线程发送信息

3、self.addEventListener('message', (e) => {

console.log('Message received from main thread:', e.data);

});

self.close(): 关闭 Web Worker 线程，Web Worker 执行完任务后通常会调用这个方法。

self.addEventListener('error', (e) => {

console.error('Error in worker:', e.message);

});

4、Web Worker 的限制

无法访问 DOM: Web Worker 运行在独立线程中, 无法直接操作 DOM 元素。

不能使用 window 和 document 对象: Web Worker 不具有 window 或 document 对象。

只能使用有限的全局对象: Web Worker 中可用的全局对象包括 self, importScripts, postMessage, close 和 ErrorEvent。

无法直接使用 localStorage: Web Worker 不支持直接使用 localStorage, 可以通过 postMessage 与主线程沟通来实现类似功能。

5、说一下 FileReader

FileReader 是 JavaScript 中的一个 Web API, 它允许 web 应用程序异步读取用户计算机上的文件 (或原始数据缓冲区) 的内容, 例如读取文件以获取其内容`。

并在不将文件发送到服务器的情况下在客户端使用它。这对于处理图片、文本文件等非常有用, 尤其是当你想要在用户界面中即时显示文件或进行文件预览时。

readAsArrayBuffer(file): 异步读取文件内容, 并将结果作为一个 ArrayBuffer 对象返回。

readAsDataURL(file): 异步读取文件内容, 并将结果作为一个表示文件内容的 DataURL 返回。`这特别适用于图片文件, 因为它可以直接被 <img> 标签的 src 属性使用`。

readAsText(file, [encoding]): 异步读取文件内容, 并将结果作为文本字符串返回。你可以指定一个编码 (默认为 'UTF-8')

onload: 当读取操作成功完成时触发。

onerror: 当读取操作发生错误时触发。

onabort: 当读取操作被中止时触发 (可以通过调用 abort() 方法来中止)。

onprogress: 在读取过程中周期性触发, 可以用来显示读取进度

onloadstart, onprogress, onloadend: 这些事件提供了读取操作的更多控制点, 比如开始、进度和结束。

6、拖拽 API

dragstart: 事件主体是被拖放元素, 在开始拖放被拖放元素时触发。

darg: 事件主体是被拖放元素, 在正在拖放被拖放元素时触发。

dragenter: 事件主体是目标元素, 在被拖放元素进入某元素时触发。

dragover: 事件主体是目标元素, 在被拖放在某元素内移动时触发。

dragleave: 事件主体是目标元素, 在被拖放元素移出目标元素是触发。

drop: 事件主体是目标元素, 在目标元素完全接受被拖放元素时触发。

dragend: 事件主体是被拖放元素, 在整个拖放操作结束时触发。

7、什么是盒子模型

1、内容 content

2、内边距 padding

3、边框 border

4、外边距 margin

标准盒模型: $\text{height} + \text{padding} + \text{border} + \text{margin} = \text{content} + \text{padding} + \text{border} + \text{margin}$

IE 盒模型: $\text{height} + \text{margin} = \text{content} + \text{padding} + \text{border} + \text{margin}$

box-sizing: content-box|border-box|inherit:

content-box 默认值, 元素的 width/height 不包含 padding, border, 与标准盒子模型表现一致

border-box 元素的 width/height 包含 padding, border, 与怪异盒子模型表现一致

也就是说 margin 始终不参与 height 和 width 的计算

inherit 指定 box-sizing 属性的值, 应该从父元素继承

8、css 有哪些选择器？优先级？那些属性可以继承？

1、id 选择器 100

2、类，伪类，属性选择器。例如 .classname、[type="text"]、:hover。

3、元素选择器

4、后代，子代相邻同胞选择器 #ab div 该 id 选择器下面的所有 div 元素

! important > 内联 > ID 选择器 > 类选择器 > 标签选择器

可继承：

1、与文字有关的样式：

2、与文本有关的属性：line-height, text-indent, text-align 行高，缩进，对齐方式等

3、元素可见性：visibility

4、列表属性：list-style-type：文字前面的小点点样式

5、表格布局属性：border-collapse：合并表格边框，border-spacing：设置相邻单元格的边框间的距离

6、光标：cursor

注意：a 标签的字体颜色不能被继承，h1-h6 标签字体的大小也是不能被继承的

9、说说 em、px、rem、vh、vw、vmin、vmax 的区别

em：相对父元素的字体大小

px：像素

rem：相对根元素字体大小，适用于全局一致的尺寸设定。

vh:相对浏览器窗口高度的大小

vw:相对浏览器窗口宽度的大小

vmin:当前视口宽度和高度的最小值

vmax:当前视口宽度和高度的最大值

`任意浏览器的默认字体高都是 16px`

10、设备像素、css 像素、设备独立像素 dip、dpr、ppi 之间的区别?

css 像素: 在没有缩放的情况下, 1css 像素等于 1 个设备独立像素

在同一个设备上, 每 1 个 CSS 像素所代表的设备像素是可以变化的 (比如调整屏幕的分辨率)

在不同的设备之间, 每 1 个 CSS 像素所代表的设备像素是可以变化的 (比如两个不同型号的手机)

`可以把 css 单位认为是一个虚拟相对的单位, 不同的显示器 css 表示的设备像素是不一定的。`

设备像素: 指设备能控制显示的最小物理单位。屏幕生产之后设备像素就是一个固定值, 单位`pt`。和分辨率挂钩

`一个 1080p 的显示器分辨率是 1920x1080, 意味着它有 1920 个水平设备像素和 1080 个垂直设备像素。`

当我们调整分辨率时, 屏幕显示的宽度和高度会变化, 而不是清晰度。`

`DIP(设备独立像素): 与设备无关的逻辑像素, 代表可以通过程序控制使用的`虚拟像素`, 是一个总体概念, 包括了 CSS 像素` (分辨率) `设备独立像素包括了 css 像素

`DPR(设备像素比): 设备像素和 CSS 像素之间的比例关系, `表示一个 CSS 像素包含多少个设备像素`.

DPR=设备像素/css 像素 `对于一个 DPR 为 2 的设备来说, 一个 CSS 像素对应 2x2 的设备像素矩阵, 也就是 4 个设备像素。`

1080 1920 分辨率对应的 css 像素为 540px 和 960px

PPI:每英寸像素, 表示每英寸包含的像素点的数目, 可以理解为像素密度, 数值越高, 说明屏幕显示的图像

越清晰。

屏幕分辨率: X Y

$PPI = \sqrt{X^2 + Y^2}$ 除以屏幕尺寸

‘屏幕的缩放会改变 dpr 的值。’

1、无缩放情况下, 1 个 CSS 像素等于 1 个设备独立像素

2、设备像素由屏幕生产之后就不发生改变, 而设备独立像素是一个虚拟单位会发生改变

3、PC 端中, 1 个设备独立像素 = 1 个设备像素 (在 100%, 未缩放的情况下)

4、在移动端中, 标准屏幕 (160ppi) 下 1 个设备独立像素 = 1 个设备像素

5、设备像素比 (dpr) = 设备像素 / 设备独立像素

6、每英寸像素 (ppi), 值越大, 图像越清晰

‘补充: 由于 pc 端的宽高不一致, 设置 px 会出现适配问题’

‘响应式设计: 是一种网络页面设计布局, 页面的设计与开发应当根据‘用户行为以及设备环境’(系统平台、屏幕尺寸、屏幕定向等)’进行相应的响应和调整’

响应式设计的基本原理是通过媒体查询检测不同的设备屏幕尺寸做处理, 为了处理移动端, 页面头部必须有 meta 声明 viewport

```
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no" >
```

属性:

width=device-width: 是自适应手机屏幕的尺寸宽度

maximum-scale: 是缩放比例的最大值

initial-scale: 是缩放的初始化

user-scalable:是用户的可以缩放的操作

11、如何解决前端屏幕适配问题

1、使用媒体查询

```
@media screen (min-width: 1200px) {
```

```
.container {
```

```
    max-width: 1140px;
```

```
    margin: 0 auto;
```

```
}
```

```
}
```

```
/* 适用于中等屏幕 */
```

```
@media screen (min-width: 992px) and (max-width: 1199px) {
```

```
.container {
```

```
    max-width: 960px;
```

```
    margin: 0 auto;
```

```
}
```

```
}
```

```
/* 适用于小屏幕 */
```

```
@media screen (max-width: 991px) {
```

```
.container {
```

```
    max-width: 720px;
```

```
    margin: 0 auto;
```

```
}  
  
}
```

2、使用 (flex, grid) 布局, 比如直接设置比例 flex: 1

3、百分比的设置 width: 80%

4、使用视口单位 vw,vh

5、使用 JavaScript 动态调整

6、使用 em 和 rem 单位

7、CSS 框架 (如 Tailwind CSS、Bulma) 提供了丰富的工具类, 帮助快速实现响应式设计

12、谈谈你对 BFC 的理解?

BFC:块级格式化上下文, 它是页面中的一块渲染区域, 并且有一套自己的渲染规则

触发条件:

1、根元素, 即 HTML 元素

2、浮动元素: float 值为 left、right `父元素设置 float 属性`

3、overflow 值不为 visible, 为 auto、scroll、hidden `除了 visible 其它都可`

4、display 的值为 inline-block、inltable-cell、table-caption、table、inline-table、flex、inline-flex、grid、inline-grid

5、position 的值为 absolute 或 fixed

6、设置 display: flow-root

`理解: 按照正常块级排列的方式来说, 有很多缺陷, 因此给块级元素内部开启 BFC 以解决这些排放的缺陷。

即给块级元素设置一个独立的排列规则`

常规块级元素排列有以下缺陷:

1、水平方向上，撑满整个包含块宽度，垂直方向上，依次摆放

2、垂直方向上相邻的元素，margin 会合并`比如第一个 div 设置 margin10px,第二个 div 设置 margin20px,会合并小的 margin，因此两者的间距为 20px` [ok]

3、父子关系情况下可能会产生 margin 塌陷`margin 塌陷：在垂直方向上，子元素和父元素的 margin 不存在(当设置了 margin 的情况下)`：此处也可以给父盒子设置 padding 或者 border 属性也可以解决 [ok]

4、父子关系情况下，父元素无视浮动元素会产生高度坍塌

`父元素没有设置高度的情况下，其高度是由子元素撑开的，但当子元素都设置了浮动元素之外，父元素无法获取子元素的高度，也就是父元素高度为 0，看不到了`

5、兄弟关系下，正常元素可能会被浮动元素覆盖（正常元素在浮动元素之后） =>可以理解为开启 BFC 后，不同的 BFC 区域渲染时不会相干扰

`浮动元素相比较正常元素层级更高，但不占有空间，当设置浮动元素后，后面的元素会被浮动元素覆盖`

注意：开启 display:absolute 这个层级比 float 属性更高，因此后面的元素覆盖浮动元素

开启 BFC

1、其子元素不再产生 margin 塌陷的问题

2、就算子元素浮动，自身高度也不会坍塌（高度计算不在无视浮动元素）

3、自身不会被其它元素覆盖

`注意：BFC 不能解决 margin 合并的问题，直接多设置了 margin 宽度即可`

13、如何清除浮动

1、当一个父容器内的所有子元素都被浮动时，父容器的高度会塌陷为 0，因为浮动元素不占据父容器的高度。

2、如果浮动元素后面有兄弟元素，这些兄弟元素会被浮动元素挤到一旁，导致布局错乱

1、在父容器上使用伪元素 `::after` 并设置 `content:"";display:block, clear: both`

2、给父容器添加 `overflow: hidden`、`overflow: auto` 或 `overflow: scroll`。

3、在浮动元素之后添加一个空的 `div` 元素，并设置 `clear: both`

4、创建一个通用的 `clearfix` 类，通过 `::after` 伪元素来清除浮动。

`12 解决父容器高度坍塌，34 解决兄弟元素排列问题`

14、元素垂直居中的方法？如果元素不定宽高？

1、flex 布局：`justify-content: center; align-items: center;`

2、grid 布局：`display: grid; place-items: center;`

3、table 布局：`display: table; 子元素设置：display: table-cell;vertical-align: middle; /* 垂直居中 */text-align: center; /* 水平居中 */`

4、使用定位和 transform：父元素 `relative` 定位，子元素 `absolute` 定位，且 `top:50%;left:50%=>` 子元素的左上角位于父元素的中间

`transform:translate(-50%, -50%);`移动自身宽高的 50% [或者使用 `margin: -50%`]

5、`margin:auto`,父元素 `display:flex`,子元素 `margin:auto`

`水平居中：`text-align:center`,垂直居中：`line-height = height``

15、如何实现两栏布局，右侧自适应？三栏布局中间自适应呢？（如何居中处理）

1、采用 flex 布局：大盒子采用 `display: flex`;左边固定宽高，右边设置 `flex-grow: 1`；三栏布局的原理一样

`flex-grow: (将剩余空间) 按照比例拓展。区别于 flex: 1.将全部空间按比例分。一般这两个属性都有其它

值。例如 `flex-grow:1`，`flex-grow:2` 即第一份占比三分之一`

flex-direction: 主轴方向

flex-wrap: 是否换行

flex-flow: (将剩余空间) 按照比例拓展

justify-content: 主轴上的对齐方式

align-items: 交叉轴的对齐方式

align-content: 定义多根轴线的对齐方式。如果项目只有一根轴线, 该属性不起作用

`item 属性 (子元素): order, 定义了 item 的排列顺序, 数字越小, 排列越靠前`

使用场景: 方便实现元素水平垂直居中, 方便自适应布局, 目前移动端、小程序开发基本都建议 Flex 布局

2、grid 布局: `.container { display: grid; grid-template-columns: 200px 1fr 200px; /* 左右固定 200px, 中间自适应 */ }` fr 表示自适应布局

`当表格内部为行内元素时: display: inline-grid`

grid-template-columns: 定义每一列的列宽

grid-template-rows: 定义每一行的行高

grid-template-areas: 用于定义区域

`简写: grid-template`

repeat(a,b) 重复的次数和重复的值

1、`justify-items: center; align-items: center;` 居中处理` flex 布局和 grid 布局都是这么处理的

2、通过定位和 transform: 先定位 50%, 50%。在移动 translate (-50%, -50%)

3、定位和 margin: 和上面的原理一样, 设置 margin-left 和 margin-top 都为负值, 缺点需要计算具体的 px 值。

4、table: 设置父元素为 display: table-cell, 子元素设置 display: inline-block。利用 vertical: middle 和

text-align: center 可以让所有的行内块级元素水平垂直居中

Grid 布局：是一种二维布局系统，用于创建复杂的网页布局。它通过行和列将页面分成网格，从而可以轻松地将内容放置在网格中的任意位置。[把 flex 布局看成一维布局]

3、定位：用 100vw 减去两边的宽度

`auto` 属性：弹性设置，也就是根据其它内容的大小综合设置。在 grid 布局中类似 fr`

width,height 属性设置 auto，意味默认宽高，块级元素自动填满父级元素，行内元素根据内容大小填。

margin:水平居中

16、怎么理解元素定位

在网页设计和开发中，文档流（NormalFlow）是指网页中元素的默认排列顺序，这种排列方式由浏览器的渲染引擎自动处理。

在未应用任何 CSS 布局技术（如浮动、定位等）时，元素遵循文档流的自然规则来显示。 -->

块级元素：在文档流中自动填满其父元素的宽度，并在其前后形成换行。常见的块级元素包括

<div>, <p>, <h1>-<h6>, <section>, <article> 等。它们默认从上到下堆叠。

内联元素：不会独占一行，只占据它所需的空间。常见的内联元素有 , <a>, , 等。

这些元素通常出现在块级元素内部，它们沿着文本方向排列。

元素尺寸：在文档流中，块级元素可以设置宽度和高度，内联元素虽然可以设置高度和宽度，但可能不会影响文档流中的布局（比如，不会推开其他文本或元素）。

理解：有关高度的设置是无效的，例如上间距和下间距，[左右之间的间距设置是有效的]

描述：

static:静态定位，按照文档流的排列方式布局

fixed：元素的位置相对于浏览器窗口固定，即使窗口被滚动，元素也会停留在指定位置。

使用场景：适用于需要固定在视口特定位置的元素，如固定头部或底部栏。

Absolute 定位：元素的位置相对于`最近的非 static [因此采用 relative 定位]定位的祖先元素`进行定位。

如果没有这样的祖先元素，则相对于初始包含块（通常是视口）。

Relative 定位：描述：元素的位置相对于其在文档流中的原始位置进行调整。设置为 relative 的元素仍保留在文档流中，即使其位置被调整。

`不会破坏文档流` relative 元素以它原来的位置为基准偏移，在其移动后，原来的位置依旧占据空间。

粘性定位：结合相对和固定定位的特性，在指定阈值范围内表现为相对定位，超出阈值后表现为固定定位。

17、 CSS3 新增了哪些新特性？

1、选择器：主要增加了属性选择器和伪类选择器（:nth-child(2);:last-child)

2、新样式：border-radius：创建圆角边框， box-shadow：为元素添加阴影， border-image：使用图片来绘制边框

3、媒体查询：@media

4、flex 和 grid 布局

5、文字：word-wrap 是否换行, text-overflow 文本溢出 clip,ellipsis 裁剪或者省略号代替, text-shadow 文本阴影

6、颜色：rgba(),rgb+a 透明度

7、增加了一些动画的效果：transition, transform, animation, linear-gradient:线性渐变, radial-gradient: 径向渐变

`transform:适用与 2d 和 3d 转换` `注意：只会触发重绘，不会触发重排`

语法：transform： 变换函数（translate,scale,rotate.skew）

translate:translateX(length),translateY,tanslateZ,tanslate(a,y),tanslate3d(x,y,z)

scale:scale(number),scaleX,scaleY,scaleZ,scale(x,y),scale3d(x,y,z)缩放比例

rotate:rotate(45deg)....

skew:skew(45deg), 只能用于 2D 倾斜

matrix:matrix(a,b,c,d,e,f)

X 轴的缩放因子、Y 轴的倾斜因子、X 轴的倾斜因子、Y 轴的缩放因子、X 轴的平移量、Y 轴的平移量。

transition: property duration timing-function delay;

`属性用于在【两个状态】之间平滑过渡。常用于元素的样式变化，如颜色、尺寸、位置等`需要事件触发，
比如 hover

property: 要应用过渡效果的 CSS 属性 (例如 width、height、background-color 等)。

duration: 过渡效果的持续时间 (例如 2s、500ms)。

timing-function: 过渡的速度曲线 (例如 linear、ease、ease-in、ease-out、ease-in-out、
cubic-bezier(n,n,n,n))。

delay: 过渡效果的延迟时间 (例如 0s、1s)。 举例定义两个类 .box{} .box:hover{} 这两个状态的变化

transition: width 2s ease 1s, height 2s ease 1s, background-color 2s ease 1s;

`animation` 属性用于定义关键帧动画，允许元素在【多个状态】之间进行复杂的动画。`不需要事件触发，
其实就是复杂动画

name: 定义动画的名称，对应于 @keyframes 规则的名称。

duration: 动画的持续时间 (例如 2s、500ms)。

timing-function: 动画的速度曲线 (例如 linear、ease、ease-in、ease-out、ease-in-out、cubic-bezier)。

delay: 动画的延迟时间 (例如 0s、1s)。

iteration-count: 动画播放的次数 (例如 infinite、1、2)。

direction: 动画的方向 (例如 normal、reverse、alternate、alternate-reverse)。

fill-mode: 动画结束时元素的状态 (例如 none、forwards、backwards、both)

选择器, 边框, 背景, 文字, 颜色, transition, transform, animation, 渐变

18、怎么理解回流和重绘? 什么场景下会触发?

opacity: 设置元素的不透明度, 会影响所有子元素的透明度, 仍然参与布局和响应鼠标事件, 只是不可见。

rgba: 其中 a 表示 alpha 通道, 取值从 0 到 1。只影响背景色, 边框色或文字, 不影响子元素的透明度

visibility: visible, hidden。不影响子元素的可见性, 元素仍然占据布局, 但不会响应鼠标事件。

display: none, 把该元素直接移除了

此处可以补充 v-if 和 v-show 的区别

回流: 当页面布局和几何属性 (大小, 位置) 发生变化时, 浏览器重新计算元素的位置和大小的过程, 回流会影响整个页面或者说页面的一部分。

触发条件: 添加删除 DOM 节点, 元素的尺寸, 边框 (宽度发生变化), 边距, 填充, 浏览器窗口的大小, 元素内容的变化等等。

offsetTop、offsetLeft、offsetWidth、offsetHeight、scrollTop、scrollLeft、

scrollWidth、scrollHeight、clientTop、clientLeft、clientWidth、clientHeight

这些属性有一个共性, 就是需要通过即时计算得到。因此浏览器为了获取这些值, 也会进行回流

重绘: 重绘是指当元素的外观 (颜色, 背景色, 边框样式) 发生变化时, 但没有影响布局, 浏览器会重新绘制元素。

19、如何实现单行 / 多行文本溢出的省略样式

单行文本溢出：

text-overflow: 规定当文本溢出时，显示省略符号来代表被修剪的文本

white-space: 设置文字在一行显示，不能换行

overflow: 文字长度超出限定宽度，则隐藏超出的内容

多行文本溢出省略：

1、伪元素 ::after 设置里面的内容 content: ...,设置父元素的高度，超过部分隐藏 overflow: hidden;, 再加上定位（基于高度截断）

2、-webkit-line-clamp: 2 （基于行数截断）

20、让 Chrome 支持小于 12px 的文字方式有哪些？区别？

客户端：用户可以直接在客户端 chrome://settings/languages 里面修改

服务端：

一、zoom：将字体大小缩放为 0.8 倍，这需要考虑兼容性

.span1{

font-size: 12px;

display: inline-block;

zoom: 0.8;

```
}
```

二、`-webkit-transform:scale()` 针对 chrome 浏览器,加 webkit 前缀,用 `transform:scale()`这个属性进行放缩

注意的是,使用 scale 属性只对可以定义宽高的元素生效,所以,下面代码中将 span 元素转为行内块元素

```
.span1{  
    font-size: 12px;  
    display: inline-block;  
    -webkit-transform:scale(0.8);  
}
```

21、css 预编译语言之间的区别

CSS: 原生支持变量、自定义属性、和部分的模块化功能。嵌套和混合功能有限。

Sass: 功能强大,支持变量、嵌套、混合、模块化,适合大型项目。

Less: 类似于 Sass,语法简洁,功能全面。

Stylus: 语法最简洁,支持变量、嵌套、混合、模块化,灵活性高。

补充: scss 相对于 css 的优势,scss 类似 css, sass 不用`

1.变量: css 语法更为繁琐

2.嵌套: CSS 目前不支持嵌套,需要手动`编写选择器`。

3.混合: CSS 没有内置的混合功能,scss 支持强大的混合功能,可以带参数(`@maxin,@include`)

4.继承: CSS 没有直接的继承功能,支持继承,减少重复代码(`@extend`)

5.运算: CSS 支持部分运算(`calc`),但不如 SCSS 灵活。 `width: 100% - 20px; padding: 10px / 2;`

6.导入: 支持`@import`,但会产生多个 HTTP 请求,影响性能,scss 支持`@import`,在编译时合并为一个文

件

7.模块系统：CSS 没有模块系统，所有变量都是全局的。支持模块系统@use 和@forward，更好地管理和组织代码。

8.条件循环：CSS 没有条件和循环功能，需要借助 JavaScript。scss 支持条件语句和循环，使样式更具动态性

9.SCSS 有强大的社区和生态系统，提供了许多工具和库（如 Compass、Bourbon），使开发更加高效。

22、如果要做优化，CSS 提高性能的方法有哪些？

1、精简样式文件：很多情况下我们会把多个样式文件合并成一个，因为通常页面之间有公共的样式

缺点：样式文件偏大，`影响首屏加载速度`。浏览器会进行多余的样式匹配，影响渲染时间。

优点：样式文件会被浏览器缓存，进入其它页面时不用下载。

2、利用继承减少代码量

3、内联首屏关键 CSS：(内联样式不能缓存，且维护困难)，内联样式能够使页面渲染速度提前因为在 HTML 下载完之后就可以开始渲染，不需要像 Link 标签引入耗费时间。一般只写一些关键 CSS 代码，如涉及整体的宽高

4、异步加载 css 代码：有些 css 代码在当前用不到的情况下异步加载

`利用 media 属性将 CSS 文件标记为非屏幕样式，当页面加载完成后再更改 media 属性以启用样式。`

```
<link rel="stylesheet" href="styles.css" media="print" onload="this.media='all'">
```

```
<noscript> <link rel="stylesheet" href="styles.css"> </noscript>
```

`使用 JavaScript 动态创建 <link> 标签并插入到页面中。`

```
<script>
```

```
var link = document.createElement('link');
```



```
link.rel = 'stylesheet';
```

```
link.href = 'styles.css';
```

```
document.head.appendChild(link);
```

```
</script>
```

5、压缩文件：借助一些打包文件进行压缩，比如 webpack,gulp/grunt

6、正确使用选择器：通配符尽量少用，不要嵌套太多的选择器，使用 id 选择器就没必要进行嵌套

7、慎用一些 CSS 属性：box-shadow,filter,:nth-child

8、慎用@import：这个会影响 css 文件的并行下载，只有当该文件下载解析完之后才能下载其它 css 文件

9、减少回流和重绘

10、优化小点：减少高性能属性浮动、定位，属性值为 0 时，不加单位。把小的 icon 图片转成 base64 编码

CSS3 动画或者过渡尽量使用 transform 和 opacity 来实现动画，不要使用 left 和 top 属性

补充：@import 和 Link 标签的区别

@import:需要在 css 文件中使用，早期的浏览器可能不支持，但会增加额外的 HTTP 请求，影响性能。

只有当解析到该 css 文件的这一行时才会进行加载文件，而且这个 css 文件是`同步单线`加载的，只有加载完这个文件才会加载下一个文件

Link:<link> 标签在 HTML 解析时立即加载 CSS 文件，并行加载，不会阻塞页面渲染。也可以通过 JavaScript 动态加载。

23、浏览器生成页面的过程(浏览器的渲染原理)

1、当浏览器接收到服务器的 HTML 响应后,开始逐行解析 HTML 标记,并构建 DOM 树,`遇到 <script>

标签时,会暂停解析 HTML,开始加载和执行 JavaScript.`，与此同时加载 CSS，浏览器会根据 HTML

中的 `<link>` 标签或嵌入的 `<style>` 标签加载并解析 CSS 文件构建 CSSOM 树。

2、构建渲染树：浏览器结合 DOM 树和 CSSOM 树，创建渲染树 (Render Tree)。

渲染树节点：渲染树中的每个节点对应一个将被绘制到屏幕上的元素。渲染树不包含隐藏的元素 (如 `display: none`)。

3、计算布局：浏览器通过遍历渲染树，计算每个节点的几何信息 (位置和大小)，确定页面元素在屏幕上的确切位置。这一步也称为回流 (Reflow)。

布局算法：浏览器使用不同的布局算法处理块级元素、行内元素、浮动元素和其他 CSS 布局特性。

4、绘制：绘制过程：浏览器遍历渲染树，将每个节点绘制到屏幕上。绘制过程包括文本、颜色、图像、边框、阴影等的呈现。

绘制顺序：浏览器按照树的顺序从上到下绘制，每个节点会被绘制成多个图层。

5、现代浏览器使用合成器线程将页面分成多个图层 (Layers)，每个图层可以独立绘制和更新：

分层：浏览器将渲染树分割成多个图层，这些图层可以独立绘制。

合成图层：浏览器的合成器线程将这些图层合成一个最终的页面。

GPU 加速：使用 GPU 加速渲染图层，提高渲染性能和流畅度。

总结：

1、HTML 解析：浏览器解析 HTML，构建 DOM 树。

2、CSS 解析：浏览器解析 CSS，构建 CSSOM 树。

3、构建渲染树：结合 DOM 树和 CSSOM 树，创建渲染树。

4、JavaScript 执行：

同步脚本：阻塞 HTML 解析，加载并执行。

异步脚本：异步加载，不阻塞 HTML 解析，加载完成立即执行。

延迟脚本：异步加载，HTML 解析完成后执行。

5、布局与绘制：计算每个节点的几何信息，绘制页面。

6、事件循环：处理异步任务和事件。

补充：async 和 defer 的区别理解：脚本的执行和 HTML 的解析不能同时进行，defer 延时，就是将脚本执行放在后面

异步脚本：通过 async 属性，脚本可以异步加载，但执行时仍会阻塞 HTML 解析[加载时 HTML 继续解析，执行时 HTML 暂停解析]

```
<script src="script.js" async> </script>
```

延迟脚本：通过 defer 属性，脚本可以在 HTML 完全解析后再执行，不会阻塞 HTML 解析
加载时 HTML 继续解析，HTML 解析完后再执行

```
<script src="script.js" defer> </script>
```

没有 defer 或 async，浏览器会立即加载并执行指定的脚本，“立即”指的是在渲染该 script 标签之下的文档元素之前，也就是说不等待后续载入的文档元素，浏览器读到脚本就加载并执行。脚本执行完，再继续解析剩下的 html 未解析的部分。

补充：<script type="module"> 标签：类似于 defer，代码是模块化的，局部作用域。支持 import 和 export。严格模式下运行，且不污染全局作用域。执行顺序根据依赖关系，不是位置的先后顺序

src 和 href 的区别

src:针对 script 标签，一般用于引入脚本，会暂停页面的解析

href:也会并行下载，但不会停止当前文档的处理，一般用于引入网络资源（图片等），针对 a 标签和 link 标签(一般用于引入 css 文件)。

24、data 对象的使用

```
const date = new Date(1695038400000);
```

```
console.log(date);

console.log(date.getFullYear()); // 获取年，例如 2024

console.log(date.getMonth()); // 获取月 (0-11)，例如 8 表示 9 月

console.log(date.getDate()); // 获取日 (1-31)，例如 18

console.log(date.getHours()); // 获取小时 (0-23)

console.log(date.getMinutes()); // 获取分钟 (0-59)

console.log(date.getSeconds()); // 获取秒 (0-59)

console.log(date.getMilliseconds()); // 获取毫秒 (0-999)

console.log(date.getDay()); // 获取星期几 (0-6)，0 表示星期天

console.log(date.getTime()); // 获取自 1970 年 1 月 1 日以来的毫秒数 (时间戳)

const year = date.getFullYear();

const month = ('0' + (date.getMonth() + 1)).slice(-2); // 补齐两位数字

const day = ('0' + date.getDate()).slice(-2);

console.log(`${year}-${month}-${day}`); // 格式化为 YYYY-MM-DD

`注意：可以直接把毫秒数放入 Date 函数里面`
```

25、比较 Link 和@import 标签

link 标签在 vue 项目中可以用于加载全局样式，比如在入口文件中加载 index.html

<link> 标签：HTML 文档在解析时，会同步加载 <link> 引入的 CSS 文件，可以同步加载多个样式表。它不会阻塞 HTML 的解析，但在样式表加载完毕之前，页面不会渲染。<link> 更适合加载外部样式表，尤其是全局样式表。

@import: 在 CSS 中使用 @import 会在页面加载完毕后才加载导入的 CSS 文件, 因此相较于 <link>,

@import 会导致样式加载得更慢。@import 的文件是在解析到该语句时才会加载, 不是立即加载的。

<link> 标签: 可以在 HTML 文件中使用, 且它不仅限于引入 CSS, 还可以用于引入其他资源, 比如网站的图标 (favicon)。

@import: 只能在 CSS 文件或 <style> 标签内使用, 并且只能导入 CSS 文件。

<link> 标签: 由于 <link> 标签引入的 CSS 文件是同步加载的, 它的样式会立即应用, 通常在解析 HTML 文档时优先加载。

@import: 由于 @import 的 CSS 文件加载较晚, 可能会影响其优先级, 在某些情况下, 样式可能会在页面完全渲染后才应用。

<link> 标签: 从最早的浏览器版本开始就广泛支持, 几乎所有现代浏览器都能良好处理 <link>。

@import: 较新的浏览器可以很好地支持 @import, 但在早期的浏览器 (如 IE5 或更旧版本) 中可能存在兼容性问题。

<link> 标签: 可以并行加载多个 CSS 文件, 浏览器会同时下载多个 <link> 标签指定的资源。

@import: 由于 @import 是在解析到该语句时才加载, 且依赖于前一个 CSS 文件的加载, 导致不能并行加载, 会引入额外的延迟。

<link> 标签: 推荐用于所有外部样式表的引入, 特别是在开发现代 Web 项目时。一般在 <head> 标签中使用 <link> 来引入外部 CSS 文件。

@import: 通常用于将多个 CSS 文件组合在一起, 或者将某些样式根据条件导入, 比如在某些特定条件下导入特定样式表。

<link> 标签: 由于是直接在 HTML 中使用的, 你可以通过 JavaScript 动态地添加或移除 <link> 标签, 从而控制样式表的加载和应用。

@import: 只能在 CSS 中使用, 无法通过 JavaScript 动态地操作, 灵活性不如 <link>。

